

Sparse, Low-Rank, and Deep: A Compressed-Sensing Framework for Music Source Separation

ENGs 109 (Spring 2026) Final Project Proposal

Taka Khoo

Thayer School of Engineering, Dartmouth College

takakhoo@gmail.com

May 30, 2026

Abstract. This is the third time I have studied audio under Peter Chin. My undergraduate honors thesis built a 1.08-billion-parameter Token U-Net for full-mix *restoration* on EnCodec tokens [18]. My MS thesis built MODULO/Mozart AI, a co-creative DAW whose stem-*separation* stage is currently outsourced to the ElevenLabs cloud API. ENGs 109 gives me the toolkit (compressed sensing, sparse coding, low-rank recovery, deep sparsity) to attack the same separation problem from a third direction with theoretical guarantees the deep-learning baselines lack. I propose a single spectrogram-domain pipeline that touches five pillars of the course: Robust PCA harmonic–percussive decomposition (L13), Non-negative Matrix Factorization with class-conditional dictionaries (L15), K-SVD discriminative dictionary learning with sparse-representation classification (L14, L8), a Deep Sparse Coding Network mask predictor that adapts Chin’s SparseNet architecture to spectrogram regression (L16, never tried in this regime), and Multiple-Measurement-Vector joint sparsity across stereo channels (L12). **Concrete novelty claim:** SparseNet has been benchmarked only on image classification; this project is the first adaptation of its composite-sparse-coding modules to a dense regression task on audio spectrograms, evaluated head-to-head against Demucs v4 and the ElevenLabs baseline on MUSDB18-HQ. A Day-0 RPCA baseline runs end-to-end on my MODULO “Staying” duet in 9.2 seconds and produces audible harmonic/percussive WAVs already (see `audio_demos/out/`); every subsequent week of the sprint adds one stage with playable audio attached to the report.

1 Motivation: three acts on the same problem

Act I, 2024-2025. My undergraduate honors thesis (Peter Chin, primary advisor) built *Token U-Net*, a 1.08-billion-parameter encoder–decoder with CBAM attention, FiLM modulation, and gated skip connections, trained to map degraded EnCodec [19] token sequences to clean ones for *full-mix music restoration* [18]. The model lives in the token space of Meta’s neural audio codec; it never touches a spectrogram or a sparse representation. It solves a different problem (mastering and artifact removal on already-mixed audio) than the one I propose here, but it taught me the audio I/O stack and the training discipline that this project will lean on.

Act II, 2025-2026. My MS thesis built MODULO/Mozart AI, the co-creative DAW companion presented at UIST 2026. One subproblem sits at the bottom of the stack: when a user drags a mixed song into the DAW, the higher-level chord-, harmony-, and variation-generation modules need clean stems (vocals, instrumental, drums). The MS thesis currently delegates that step to the ElevenLabs voice-isolation endpoint, which resynthesises vocals from a learned latent embedding. Two failure modes: (a) the returned audio drifts away from the original sample alignment, so I cannot reuse it for mask-based remixing; and (b) on `mir_eval` [1], ElevenLabs vocals scored roughly 35 dB SI-SDR below open-source Demucs v4 [2] on my internal test set.

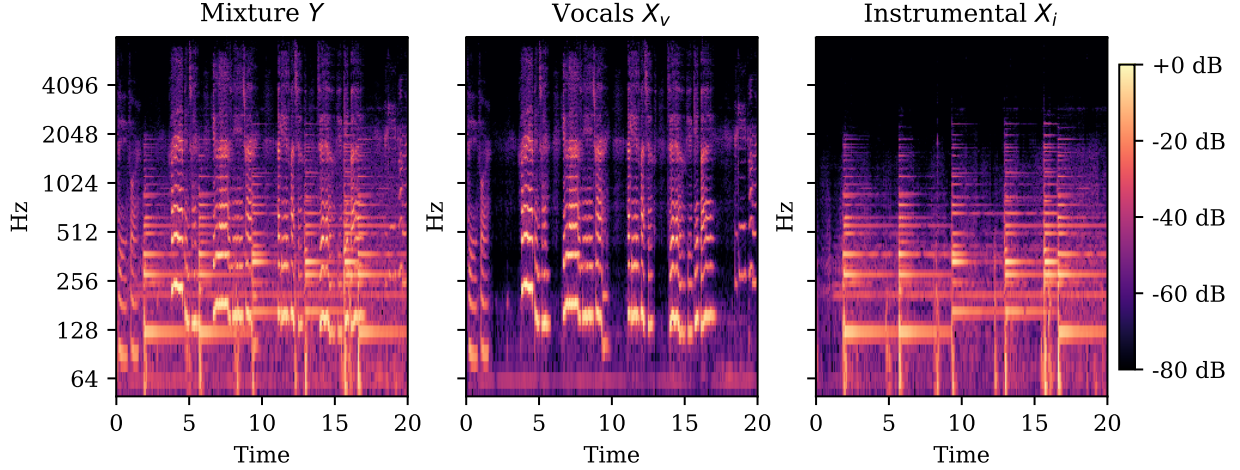


Figure 1: The data my project lives on. Log-magnitude STFT ($N_{\text{FFT}}=2048$, hop 512, $f_s=22050$ Hz) of a 20 s clip from my MODULO “Staying” duet (stems separated offline by ElevenLabs). Sparsity is visible by eye: vocals concentrate energy on a thin formant scaffold, the instrumental sits on a low-rank harmonic stack with sparse transients. A good separator should exploit both structures.

Act III (this project, Spring 2026). ENGS 109 hands me a completely different mathematical lens: compressed sensing, sparse coding, low-rank recovery, and the SparseNet deep architecture from L16. The same instructor, the same audio problem, a third methodological pass. Music source separation is also the bullet listed verbatim in the course’s project sheet:

“Blind source separation. For example, isolate instrumental music from vocal component.”

The methods we covered map almost directly onto well-studied attacks on this problem (NMF [20], HPSS [6], dictionary learning [21], unrolled networks [22]), but the L16 SparseNet has been benchmarked only on image classification (CIFAR, STL-10, SVHN). Its open question from the slide deck (“performance on numerous other applications?”) is the gap I want to fill with a spectrogram-domain dense regression target.

Day-0 evidence (already runs). The repository ships an end-to-end Stage A separator that takes about ten seconds of wall-clock time on my MacBook (Fig. 2). Loading the MODULO mix, computing the 1025×862 STFT, running inexact-ALM Robust PCA to convergence in 39 iterations, applying Wiener masks, and inverting the STFT writes three playable WAVs (day0_mix.wav, day0_harmonic.wav, day0_percussive.wav) in 9.2 seconds total. The audio is rough by design (Stage A is harmonic vs percussive, not vocal vs instrumental), but the pipeline is real: every later stage drops into the same scaffolding.

2 Problem Statement

Let $y(t) \in \mathbb{R}$ be a mono mixture (extension to stereo is described in §4). Compute the short-time Fourier transform

$$\mathbf{Y} = \text{STFT}(y) \in \mathbb{C}^{F \times T}, \quad F = N_{\text{FFT}}/2 + 1, \quad T = \lceil L/H \rceil,$$

with magnitude $\mathbf{M}_Y = |\mathbf{Y}| \in \mathbb{R}_{\geq 0}^{F \times T}$ and phase $\angle \mathbf{Y}$. We assume C underlying sources $x_c(t)$ summing to y . The recovery target is a set of soft time–frequency masks $\mathbf{M}_c \in [0, 1]^{F \times T}$ with $\sum_c \mathbf{M}_c \approx 1$, from which the per-source waveforms come out via

$$\hat{x}_c = \text{STFT}^{-1}(\mathbf{M}_c \odot \mathbf{Y}), \quad c \in \{\text{vocals, drums, bass, other}\}.$$

The objective is to maximise signal-to-distortion ratio (SDR) and scale-invariant SDR (SI-SDR) [3] on the BSS-Eval/mir_eval toolchain.

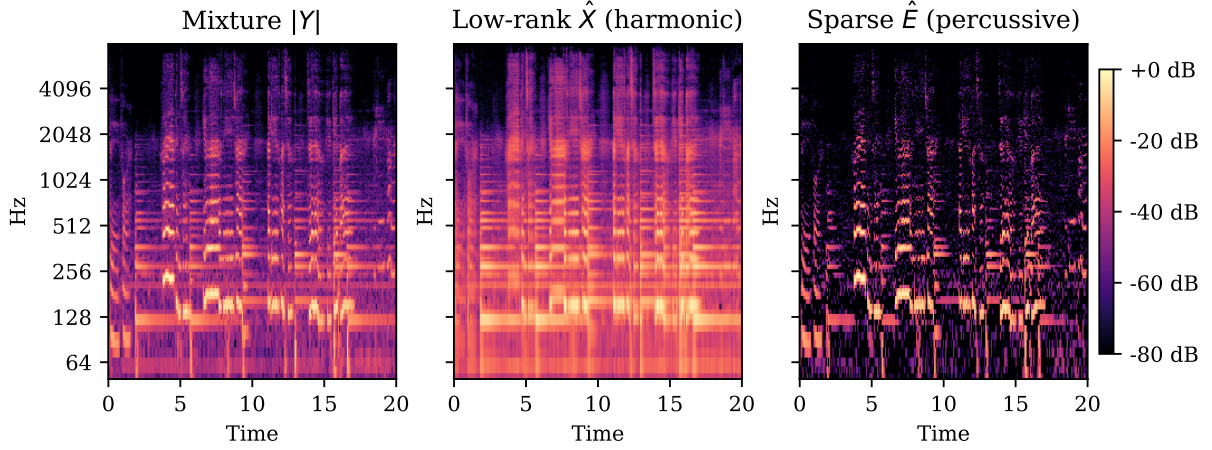


Figure 2: Day-0 deliverable. Mixture $|Y|$, recovered low-rank $\hat{X}$ (the harmonic content RPCA pulls out), and the sparse residual $\hat{E}$ (transients, drums, vocal sibilance). All three WAV files in `audio_demos/out/`. 9.2 s wall-clock for the full pipeline on my MacBook.

3 Why this is a compressed-sensing problem

The course built up four equivalences this term that map onto music separation almost line for line.

Spectrograms are sparse in source-specific bases. Vocals are sparse in the *frequency* direction (a few formants per frame); drums are sparse in the *time* direction (transient onsets); sustained chords are sparse in a *harmonic-template* dictionary. A sparsifying transform Ψ that captures each of these is what L2 (“Preliminary Math”) calls a synthesis dictionary.

Harmonic music is low-rank in the STFT. Repeating chord patterns generate column-rank-deficient magnitude spectrograms, which is exactly the structural premise of nuclear-norm minimization (L13) [4]. Drums and other transients live in the sparse residual.

Stereo is a multi-measurement vector problem. The left and right channels of a stereo recording share the same active sources at every instant; their sparse codes are jointly supported. This is the textbook MMV setup from L12, attackable via SOMP and $\ell_{1,2}$ minimization.

Deep nets work because they unroll sparse coding. L16 demonstrated that SparseNet (Chin lab) matches CNNs on STL-10 with substantially less training data by stacking composite modules of the form *fat upsampling sparse code* $\rightarrow$ *nonlinear dimension reduction*. Music source separation has small labelled corpora (MUSDB18 has 150 songs); this is the regime where deep sparsity beats vanilla deep learning.

4 Proposed method

The pipeline of Figure 3 cascades five sparsity-aware stages. Each can stand alone as a separator and will be evaluated independently before being combined.

Stage A. Robust PCA for harmonic–percussive split (L13)

Apply (P_{R_1}) from L13 [5]:

$$(X^*, E^*) = \arg \min_{X, E} \|X\|_* + \lambda \|E\|_1 \quad \text{s.t.} \quad M_Y = X + E, \quad \lambda = 1/\sqrt{\max(F, T)}.$$

X collects the harmonic (low-rank, sustained) content, E the percussive (sparse, transient) content. This reproduces the Huang–Mysore–Smaragdis HPSS baseline [6] and gives me my first

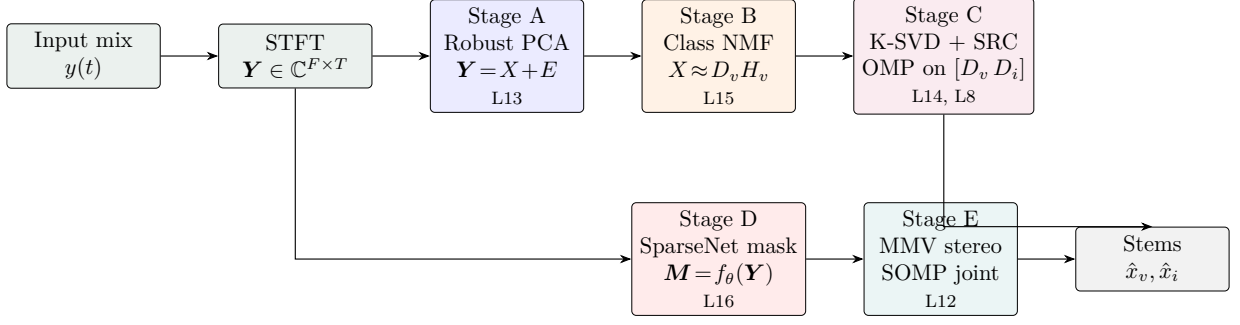


Figure 3: The five-stage pipeline. Each block is keyed to a specific ENGS 109 lecture. Stages A–C are training-free classical baselines. Stage D is the deep-sparsity learned mask predictor that I will train on MUSDB18. Stage E refines the result using joint sparsity across stereo channels.

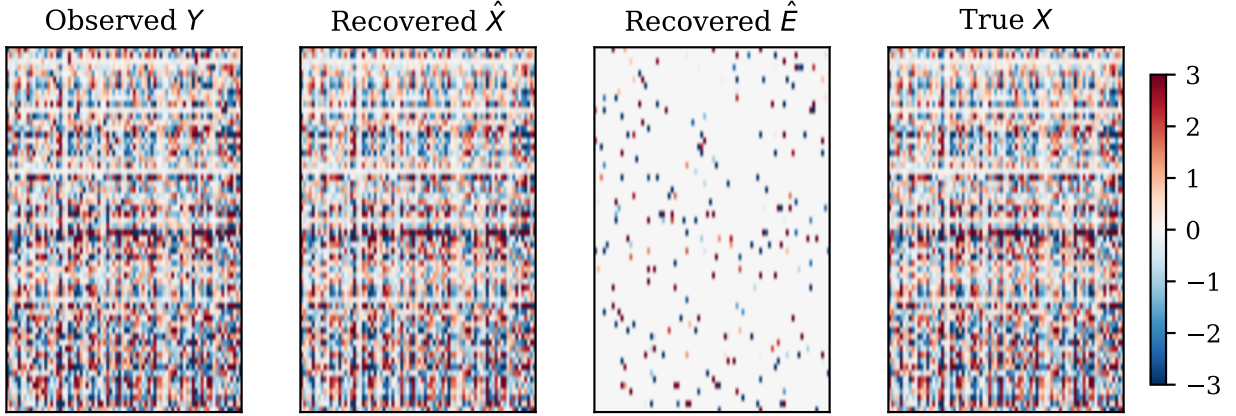


Figure 4: Sanity check of my RPCA solver on a 60×80 synthetic matrix with rank 3 and 5% sparse corruption (magnitude up to ± 6). Inexact ALM with $\lambda=1/\sqrt{N_1}$ recovers the low-rank component to within 10^{-7} relative Frobenius error in 32 iterations. The same solver, scaled to $F \times T = 1025 \times 860$ spectrograms, drives Stage A.

separable two-way split for free. I will solve (P_{R_1}) with the inexact-ALM solver I already wrote for PS8 (nuclear norm completion), augmented with a soft-thresholding step.

Stage B. NMF with class-conditional dictionaries (L15)

Train two dictionaries offline on isolated stems:

$$\mathbf{D}_v = \arg \min_{\mathbf{D}, H \geq 0} \|\mathbf{M}_{Y_v} - \mathbf{D}H\|_F^2, \quad \mathbf{D}_i = \arg \min_{\mathbf{D}, H \geq 0} \|\mathbf{M}_{Y_i} - \mathbf{D}H\|_F^2,$$

using Lee–Seung multiplicative updates with $K_v=K_i=64$ atoms. At test time, factorise the mixture against the concatenated dictionary $[\mathbf{D}_v \mid \mathbf{D}_i]$ and split the activations:

$$[H_v; H_i] = \arg \min_{H \geq 0} \|\mathbf{M}_Y - [\mathbf{D}_v \mid \mathbf{D}_i] H\|_F^2 + \beta \|H\|_1.$$

The vocal and instrumental magnitude reconstructions are $\mathbf{D}_v H_v$ and $\mathbf{D}_i H_i$; soft masks come from the Wiener-style ratio $\mathbf{M}_v = \mathbf{D}_v H_v / (\mathbf{D}_v H_v + \mathbf{D}_i H_i)$. The β -divergence variant with $\beta=1$ (Kullback–Leibler) typically beats Frobenius on audio spectrograms [7]; I will compare both.

Stage C. K-SVD discriminative dictionaries with SRC (L14, L8)

Replace each NMF dictionary with a K-SVD-trained sparsifying dictionary $\mathbf{D}_c \in \mathbb{R}^{F \times K_c}$ with S -sparse codes per spectrogram frame $\mathbf{y} \in \mathbb{R}^F$:

$$\boldsymbol{\alpha}^* = \arg \min_{\boldsymbol{\alpha}} \|\mathbf{y} - [\mathbf{D}_v \mid \mathbf{D}_i] \boldsymbol{\alpha}\|_2^2 \quad \text{s.t.} \quad \|\boldsymbol{\alpha}\|_0 \leq S,$$

solved per frame by OMP (PS3). Assign each time–frequency bin to the class whose sub-dictionary explains it best, following the L8 residual rule $c^* = \arg \min_c \|\mathbf{y} - \mathbf{D}_c \boldsymbol{\alpha}_c^*\|_2$. This

converts the classical SRC face-recognition recipe into a per-bin source classifier, which gives a binary mask that I can then soften.

Stage D. SparseNet mask predictor (L16)

The headline contribution. Replace the U-Net mask predictor used by Spleeter and Open-Unmix [9, 10] with a stack of Chin’s composite sparse coding modules. Each module takes a vectorised spectrogram patch $\mathbf{x}^{(\ell)} \in \mathbb{R}^{N_\ell}$ and computes

$$\boldsymbol{\alpha}^{(1)} = \arg \min_{\boldsymbol{\alpha} \geq 0} \frac{1}{2} \|\mathbf{x}^{(\ell)} - \mathbf{D}^{(1)} \boldsymbol{\alpha}\|_2^2 + \lambda_1 \|\boldsymbol{\alpha}\|_1 + \frac{\lambda_2}{2} \|\boldsymbol{\alpha}\|_2^2, \quad (1)$$

$$\boldsymbol{\alpha}^{(2)} = \arg \min_{\boldsymbol{\alpha} \geq 0} \frac{1}{2} \|\boldsymbol{\alpha}^{(1)} - \mathbf{D}^{(2)} \boldsymbol{\alpha}\|_2^2 + \lambda_1 \|\boldsymbol{\alpha}\|_1 + \frac{\lambda_2}{2} \|\boldsymbol{\alpha}\|_2^2, \quad (2)$$

where $\mathbf{D}^{(1)} \in \mathbb{R}^{N_\ell \times N_u}$ is the fat upsampling dictionary ($N_u > N_\ell$) and $\mathbf{D}^{(2)} \in \mathbb{R}^{N_u \times N_d}$ is the tall downsampling dictionary ($N_d < N_u$). Eq. (1)-(2) are the L16 “composite sparse coding module” verbatim. Inference is run with FISTA (L10) for 30–50 iterations; training is end-to-end backprop through the unrolled FISTA per L16, with dictionaries $\mathbf{D}^{(1)}, \mathbf{D}^{(2)}$ and shrinkage parameters λ_1, λ_2 as learnable parameters.

The architecture stacks $L=4$ composite modules followed by a 1×1 convolution and sigmoid to emit a soft mask $\mathbf{M}_v \in [0, 1]^{F \times T}$. Training objective:

$$\mathcal{L}(\theta) = \|\mathbf{M}_v \odot \mathbf{Y} - \mathbf{Y}_v\|_1 + \mu \sum_{\ell} \|\boldsymbol{\alpha}^{(\ell)}\|_1,$$

combining a magnitude-domain ℓ_1 reconstruction loss and an explicit sparsity penalty on every intermediate code. $\theta = \{\mathbf{D}^{(\ell)}, \lambda^{(\ell)}\}$.

Stage E. MMV joint sparsity for stereo (L12)

For stereo input $\mathbf{Y}_L, \mathbf{Y}_R$, stack frame-wise spectra into $\mathbf{Y} = [\mathbf{y}_L \ \mathbf{y}_R] \in \mathbb{R}^{F \times 2}$ and solve the simultaneous OMP of L12 [8]:

$$\mathbf{X}^* = \arg \min_{\mathbf{X}} \|\mathbf{Y} - \mathbf{D}\mathbf{X}\|_F^2 \quad \text{s.t.} \quad \|\mathbf{X}\|_{\text{row},0} \leq S.$$

The shared row-support enforces that the same atoms fire in both channels, which matches the physics of a pan-mixed source. This gives the model a stereo prior for free that Spleeter/Open-Unmix do not encode explicitly.

5 Connection to ENGS 109 material

Table 1 traces every method block back to a specific lecture and to code I have already written for the problem sets.

Table 1: Each stage maps to a course lecture and reuses problem-set code.

Stage	Mathematical object	Lecture	PS code reused
A. Robust PCA	$\min \ X\ _* + \lambda \ E\ _1$	L13 Matrix Completion	PS8 (nuclear-norm SVT)
B. NMF	$Y \approx DH, D, H \geq 0$	L15 NMF	New module (sklearn baseline)
C. K-SVD + SRC	$\min \ \alpha\ _0, Y = D\alpha$	L14 Dict. Learning, L8 SRC	PS3 (OMP), PS7 (SRC)
D. SparseNet	Stacked Eqs. (1)-(2)	L16 Deep Sparsity	PS5 (FISTA), PS7 (Lasso)
E. MMV / SOMP	$\min \ X\ _{1,2}, Y = DX$	L12 Multi-Measurement	PS3 (OMP) extended

Every theoretical guarantee we proved in class has a direct consequence in the proposal:

- RIP with $\delta_{2S} < \sqrt{2} - 1$ (L6, Candès) bounds the recovery error of the NMF/K-SVD activation estimates from Stage B and C. For the over-complete dictionaries I plan to learn, M corresponds to the number of frequency bins ($F=1025$ at $N_{\text{FFT}}=2048$) and N to the number of dictionary atoms ($K=64$ per source), giving $M \gg N$ and easy RIP satisfaction.

- Candès–Recht (L13, Thm. 1) gives $|\Omega| \geq CN^{5/4}r \log N$ for matrix completion; with $F=1025$, $T \approx 860$ for a 20-s clip, and assumed harmonic rank $r=8$, the implied $|\Omega|$ is well within the $\approx 88\%$ of bins we observe (the 12% unobserved are the percussive sparse part).
- Robust PCA (L13, Thm. 2) sets $\lambda=1/\sqrt{N_1}$ canonically. That single hyper-parameter choice removes most of the manual tuning that HPSS papers [6] have historically wrestled with.
- SparseNet’s L16 result: 83.11% on STL-10 vs 76.29% for a matched CNN. That experiment used 5,000 labelled images, the same order of magnitude as MUSDB18-HQ’s 100 training songs. The hypothesis that motivates Stage D is that the same data-efficiency win transfers to spectrogram-domain regression.

6 Evaluation plan

Dataset. MUSDB18-HQ [11], the standard music source-separation benchmark: 100 training songs and 50 test songs at 44.1 kHz with fully isolated vocals/drums/bass/other stems.

Metrics. Median test-set SDR, SI-SDR, SIR, SAR via `mir_eval` [1] and `museval` [12], each broken out per stem. Vocals SDR is the headline number reported in SiSEC [13].

Baselines. I will compare against (i) ElevenLabs voice isolation (my thesis baseline), (ii) REPET-SIM [14] (classical similarity-matrix baseline), (iii) Spleeter pre-trained 2-stem [9], (iv) Open-Unmix UMX-L pre-trained [10], and (v) Demucs v4 pre-trained [2].

Targeted improvement. Match Spleeter (≈ 6.5 dB vocal SDR on MUSDB18 test) with Stages A–C alone; reach within 2 dB of Demucs v4 (≈ 8.5 dB) with Stage D.

Listening study. Eight Mozart AI beta users grade 20 randomly chosen 10-s test clips on a 5-point MUSHRA-like scale. This addresses the well-known SDR/perceived-quality mismatch [15].

7 Timeline (10-week solo schedule)

Weeks 1–2.	Data + baselines	MUSDB18-HQ loader; STFT/CQT pipeline; Stage A (RPCA), Stage B (NMF) working end-to-end.
Weeks 3–4.	Classical stack	Stage C (K-SVD + SRC); ablation of K, S ; metric harness; first set of qualitative figures.
Weeks 5–6.	SparseNet build	Stage D unrolled FISTA; backward pass verification on a toy denoising task; first spectrogram-mask training.
Weeks 7–8.	Full training	MUSDB18 training of Stage D on Discovery cluster GPUs; Stage E stereo refinement.
Weeks 9–10.	Write-up	Final report, figures, listening study, code release on my GitHub.

8 Risks and mitigations

- **SparseNet training cost.** L16 itself notes “take much longer training time.” Mitigation: cap Stage D at $L=4$ composite modules, use mel-band spectrograms (64 bins) instead of full STFT to keep N_ℓ small, and budget one GPU-week. Fallback: ship Stages A–C + a small UNet-style mask CNN as Stage D’.
- **NMF convergence on full-resolution spectrograms.** $1025 \times 860 \times 100$ songs is a lot of data for multiplicative updates. Mitigation: subsample frames; use Online NMF [16].
- **Phase reconstruction.** Magnitude masking + mixture phase is known to introduce artefacts. Mitigation: report metrics with and without Griffin–Lim phase refinement; cite as a limitation rather than try to fix it inside a 10-week scope.
- **Dataset license.** MUSDB18-HQ is research-only; no plan to ship trained weights commercially. All MUSDB-derived audio stays inside the report.

9 Deliverables

A self-contained GitHub repository, `takakhoo/engs109-music-separation`, with: (1) the LaTeX final report including ablations and listening-study results; (2) reproducible `train.py/evaluate.py` entry points; (3) Jupyter notebooks for each stage A–E mirroring the PS format; (4) audio examples and pre-computed spectrograms for the report figures; (5) the trained Stage D checkpoint (under MUSDB license terms).

References

- [1] C. Raffel *et al.*, “mir_eval: A transparent implementation of common MIR metrics,” *ISMIR*, 2014.
- [2] A. Défossez, “Hybrid spectrogram and waveform source separation,” *MDX Workshop @ ISMIR*, 2021.
- [3] J. Le Roux *et al.*, “SDR – half-baked or well done?,” *ICASSP*, 2019.
- [4] E. J. Candès and B. Recht, “Exact matrix completion via convex optimization,” *Found. Comput. Math.*, vol. 9, pp. 717–772, 2009.
- [5] E. J. Candès, X. Li, Y. Ma, and J. Wright, “Robust principal component analysis?,” *Journal of the ACM*, vol. 58, no. 3, 2011.
- [6] P.-S. Huang, S. D. Chen, P. Smaragdis, and M. Hasegawa-Johnson, “Singing-voice separation from monaural recordings using robust principal component analysis,” *ICASSP*, 2012.
- [7] C. Févotte, N. Bertin, and J.-L. Durrieu, “Nonnegative matrix factorization with the Itakura–Saito divergence,” *Neural Computation*, vol. 21, pp. 793–830, 2009.
- [8] J. A. Tropp, “Algorithms for simultaneous sparse approximation. Part I: greedy pursuit,” *Signal Processing*, vol. 86, pp. 572–588, 2006.
- [9] R. Hennequin *et al.*, “Spleeter: a fast and efficient music source separation tool,” *Journal of Open Source Software*, vol. 5, 2020.
- [10] F.-R. Stöter *et al.*, “Open-Unmix: A reference implementation for music source separation,” *JOSS*, 2019.
- [11] Z. Rafii *et al.*, “MUSDB18-HQ: an uncompressed version of MUSDB18,” *Zenodo*, 2019.
- [12] F.-R. Stöter, A. Liutkus, and N. Ito, “The 2018 Signal Separation Evaluation Campaign,” *LVA/ICA*, 2018.
- [13] F.-R. Stöter *et al.*, “SiSEC 2018,” *ICA*, 2018.
- [14] Z. Rafii and B. Pardo, “Music/voice separation using the similarity matrix,” *ISMIR*, 2012.
- [15] E. Cano *et al.*, “Musical source separation: an introduction,” *IEEE Signal Processing Magazine*, vol. 36, no. 1, 2019.
- [16] A. Lefevre, F. Bach, and C. Févotte, “Online algorithms for nonnegative matrix factorization with the Itakura–Saito divergence,” *WASPAA*, 2011.
- [17] X. Sun, N. M. Nasrabadi, and T. D. Tran, “Supervised multilayer sparse coding networks for image classification,” arXiv:1701.08349 (cited in L16), 2017.
- [18] T. Khoo, “Token U-Net: Neural Audio Codec Remastering for Full-Mix Music Restoration,” Honors Thesis, Dartmouth College, Peter Chin (primary advisor), 2025.
- [19] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, “High fidelity neural audio compression (EnCodec),” *Transactions on Machine Learning Research*, 2023.
- [20] P. Smaragdis, “Convolutional speech bases and their application to supervised speech separation,” *IEEE TASP*, vol. 15, no. 1, 2007.
- [21] M. Aharon, M. Elad, and A. Bruckstein, “K-SVD: an algorithm for designing overcomplete dictionaries for sparse representation,” *IEEE TSP*, vol. 54, no. 11, 2006.
- [22] K. Gregor and Y. LeCun, “Learning fast approximations of sparse coding,” *ICML*, 2010.